

LABORATORIO INTERDISCIPLINARE B

THE KNIFE

MANUALE TECNICO

CORSO INFORMATICA F04

ANNO 2025-2026

Presentato da:

Devi Atti

Zribi Khalil

INTRODUZIONE

Che cos'è?

TheKnife è un progetto del laboratorio interdisciplinare A e B del corso di informatica dell'università degli studi dell'Insubria.

TheKnife è un programma che permette ad utenti registrati, e non, di cercare dei ristoranti nelle vicinanze e in luoghi specifici, e ai ristoratori di aggiungere i propri ristoranti e di farli trovare! L'applicazione permette anche di interagire con le esperienze passate degli altri utenti tramite le recensioni. Gli utenti registrati possono esprimere il loro pensiero, mentre i ristoratori possono ringraziarli!

Ambiente di sviluppo del progetto

Questo progetto è stato sviluppato su sistema operativo Windows 11 ed il linguaggio utilizzato è Java con JDK 21, ma il programma è stato progettato in modo tale da essere compatibile anche con versioni meno recenti e/o più recenti di Java. Per quanto riguarda la gestione dei dati è stato utilizzato il DBMS di PostgreSQL e per quanto riguarda la comunicazione tra server e client, è stato utilizzato RMI così da poter gestire la concorrenza in maniera efficiente. Per quanto riguarda la parte grafica è stato utilizzato JavaFX.

Al momento solo il server può essere utilizzato in altri sistemi operativi (sottolineo che il programma è stato testato unicamente su sistema operativo Windows 11), poiché presenza delle librerie JavaFX, Maven implementa direttamente le dipendenze necessarie per far funzionare il programma sul dispositivo in cui è stato sviluppato.

Ruoli del gruppo

- **Project manager Khalil Zribi:** definisce gli obiettivi e i risultati attesi del progetto, pianifica le attività e i compiti per ogni attività, progettandone le opportune tempistiche e scadenze.

- **System architect Khalil Zribi:** definisce e progetta i componenti del sistema software.
- **Design manager Atti Devi:** definisce e progetta la logia del sistema.
- **Document & quality manager Atti Devi:** definisce e realizza la documentazione del sistema.

STRUTTURA DEL PROGETTO

In questa sezione verranno analizzate le classi principali del progetto che saranno suddivisi in due categorie (rispettivamente i due package principali): theKnife-Client, theKnife-Server.

theKnife-Client

Il Client del progetto contiene tutta la parte relativa alla grafica del programma, la quale è stata suddivisa in due gruppi: user (utente registrato e ospite), restaurateur (ristoratore). L'utente registrato, una volta eseguito l'accesso, si troverà a disposizione i ristoranti più vicini alla sua posizione, e potrà: cercarne altri (facoltativamente anche filtrando la ricerca), visitare i suoi preferiti, visitare i ristoranti recensiti e lasciare recensioni. L'utente ospite potrà cercare dei ristoranti per un luogo specifico e visualizzare le recensioni relative. Il ristoratore potrà visualizzare i propri ristoranti, aggiungerne dei nuovi, visualizzare le recensioni relative ai propri ristoranti e rispondere ad esse.

Classe Launcher

La classe Launcher si occupa essenzialmente del lancio dell'applicazione, chiamando la classe App.

Classe App

La classe App si occupa di inizializzare il motore grafico e di aprire la finestra ipConfig.fxml gestita dalla classe IpConfigController.

Classe IpConfigController

La classe IpConfigController si occupa di ottenere i dati in input scritti dall'utente, l'indirizzo del server, e di creare la connessione con il server,

eseguendo un controllo sulla correttezza dell'indirizzo. Successivamente, stabilita la connessione con il server, carica la pagina principale del programma `page.fxml` gestita dalla classe `PageController`.

Classe `PageController`

La classe `PageController` permette di facilitare il caricamento di pagine successive tramite il metodo `selectPage(String fileName)`, e inizialmente carica la homepage. Inoltre, si occupa di impostare alcuni valori testuali della homepage dell'applicazione.

Classe `ToolbarController`

La classe `ToolbarController` si occupa di gestire la toolbar posizionata nella parte superiore della finestra, e permette di uscire dall'applicazione, minimizzare la finestra, passare tra le diverse funzioni di accesso.

Classe `HomeController`

La classe `HomeController` rappresenta la homepage e permette di selezionare il metodo di accesso e di eseguirlo.

Classe `SigninController`

La classe `SigninController` si occupa di gestire l'accesso dell'utente e di verificarne le credenziali. Infine, l'utente viene portato alla sua homepage.

Classe `SignupController`

La classe `SignupController` si occupa di gestire la registrazione dell'utente, di verificarne la validità delle credenziali e di eseguire un accesso automatico una volta confermata la corretta registrazione. Infine, l'utente viene portato alla homepage dell'utente registrato in base al ruolo scelto.

Classe `GuestController`

La classe `GuestController` si occupa di gestire l'accesso per un utente ospite, facendo scegliere all'utente un luogo in cui cercare ristoranti e lo porta alla homepage dell'utente.

Classe `UserHomeController`

La classe `UserHomeController` gestisce la homepage dell'utente e fornisce la maggior parte dei metodi che permettono all'utente di eseguire le azioni che

desidera. Questa classe gestisce anche la homepage dell'utente non registrato, in quanto, ad un utente ospite, gli è permesso eseguire le operazioni più basilari di un utente registrato, vengono semplicemente nascosti i bottoni che permettono all'utente ospite di raggiungere parti che non dovrebbe. In questo modo viene ridotta la complessità del codice.

Classe RightMenuSearchController

La classe RightMenuSearchController lavora insieme alla classe precedente, fornendo un menu laterale alla homepage, per migliorare la UX dell'utente. Anche qui, la classe viene utilizzata anche dall'utente ospite, semplicemente rendendo non presenti i bottoni che permettono all'utente ospite di fare azioni che non dovrebbe.

Classe RestaurateurHomeController

La classe RestaurateurHomeController gestisce la homepage del ristoratore e fornisce la maggior parte dei metodi che permettono al ristoratore di eseguire le azioni che desidera.

Classe ErrorContainer

La classe ErrorContainer si occupa di gestire la visualizzazione grafica di un errore di input dell'utente.

Classe ServerConnection

La classe ServerConnection si occupa di cercare il server richiesto, e di stabilire la connessione effettiva.

theKnife-Server

Il Server del progetto contiene tutta la parte relativa alla configurazione e inizializzazione del database e di effettuare richieste di dati, e ottenerli, dal database.

Classe ServerMain

La classe ServerMain si occupa di ottenere i dati di configurazione e inizializzazione del database, stabilisce una connessione, popola il database, e infine si mette in ascolto in attesa che un client si connetta.

Classe Database

La classe Database contiene il codice vero e proprio, chiamato dalla classe ServerMain per la connessione e configurazione del database PostgreSQL.

Classe Databaselnit

La classe Databaselnit legge dal file theKnifeSchema.sql, che contiene lo schema scheletro del database, e lo carica sul database.

Classe Populate

La classe Populate contiene i metodi veri e propri, richiamati da ServerMain, per il popolamento del database. Si occupa di popolare i ristoranti dal dataset di ristoranti presente e di inserire due utenti standard: un utente e un ristoratore.

Classi BookmarkQ, RestaurantQ, ReviewQ, UserQ

La classe BookmarkQ contiene le query da mandare al database riguardanti la parte relativa ai ristoranti preferiti dell'utente. RestaurantQ, invece, contiene le query riguardanti le informazioni riguardanti i ristoranti., ReviewQ, invece, contiene le query relative alle informazioni delle recensioni, mentre UserQ quelle riguardanti l'utente.

Classe ServerInterface

La classe ServerInterface rappresenta l'interfaccia del server che permette di far vedere quali sono i metodi del server che vengono resi pubblici e utilizzabili dal client.

Classe ServerImpl

La classe ServerImpl contiene l'implementazione effettiva dei metodi pubblici del server.

Classe Restaurant

La classe Restaurant rappresenta l'oggetto ristorante e permette di ottenere le informazioni specifiche di esso.

Classe User

La classe User rappresenta l'oggetto utente e permette di ottenere le informazioni specifiche di esso.

REPORT TECNICO DELLA SOLUZIONE SVILUPPATA

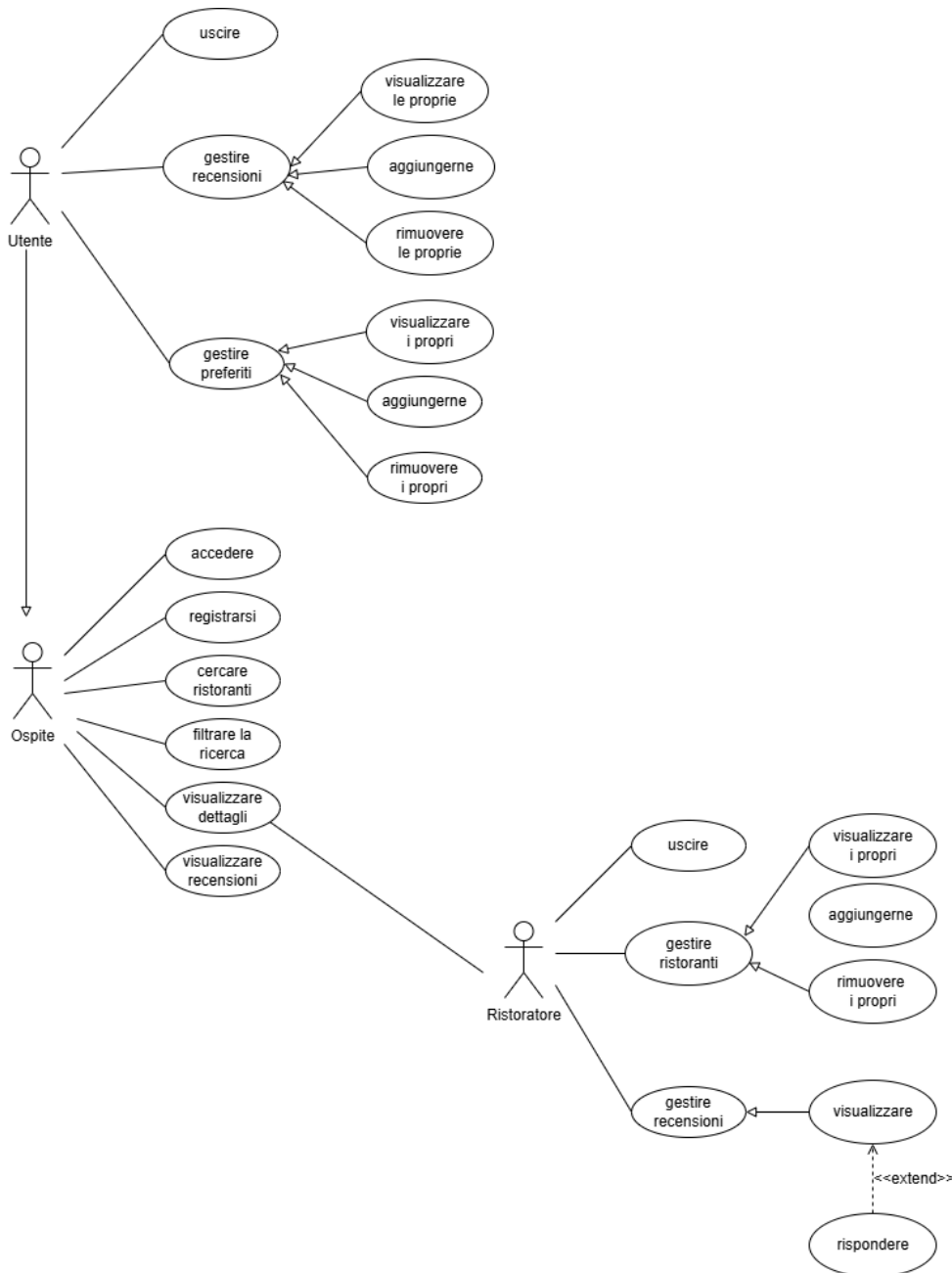
Librerie utilizzate

Per lo sviluppo di questa applicazione, sono state integrate delle librerie esterne a quelle di default offerte da Java. Per quanto riguarda il lato client, la libreria utilizzata è stata “JavaFX” che permette di sviluppare le interfacce grafiche (GUI) in Java. Mentre sul lato server è stata usata la libreria “PostgreSQL” la quale non è propriamente una singola libreria ma un insieme di driver e “API client” che consentono al linguaggio di programmazione di poter comunicare e svolgere delle operazioni sul database. Inoltre sono state utilizzate le librerie “Jbcrypt” (hashing della password dell’utente), “Common-csb” (lettura del dataset di ristoranti) e “Json” (lettura del json di risposta di openstreetmap per la traduzione di un indirizzo in termini di coordinate latitudinali e longitudinali).

Software design

La progettazione dell’applicazione è stata sviluppata tramite una serie di diagrammi UML (Unified Modeling Language) che ha consentito di definire la visione statica e dinamica del sistema.

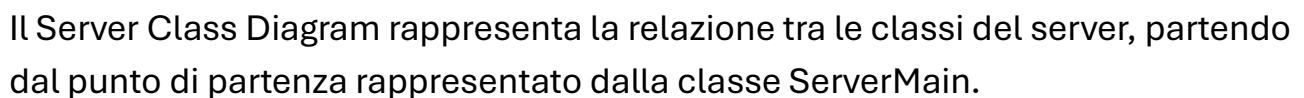
Tramite lo Use Case Diagram vengono mostrate tutte le funzionalità che TheKnife offre all’utente che lo sta usando; infatti, può eseguire l’accesso (tramite account oppure no) o effettuare la registrazione, ricercare un ristorante, visualizzare e modificare (che include sia la modifica, che l’eliminazione, che la risposta) recensioni, ristoranti preferiti, ristoranti propri.

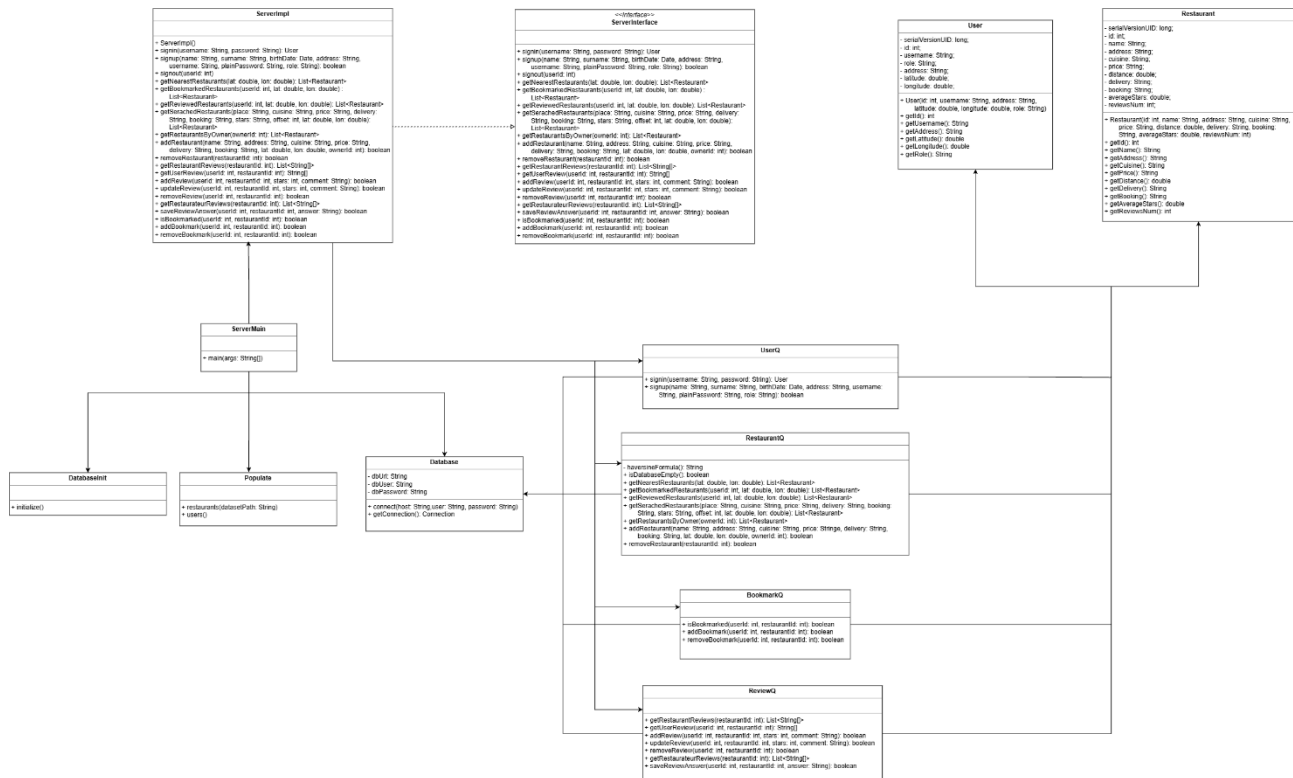


Il Class Diagram è la rappresentazione statica delle classi presenti all'interno dell'applicazione TheKnife. Sono stati creati due Class Diagram differenti ognuno corrispondente ai relativi package descritti nella struttura del progetto.

Il Client Class Diagram rappresenta la relazione tra le classi del client, partendo dal punto di partenza rappresentato dalla classe Launcher.

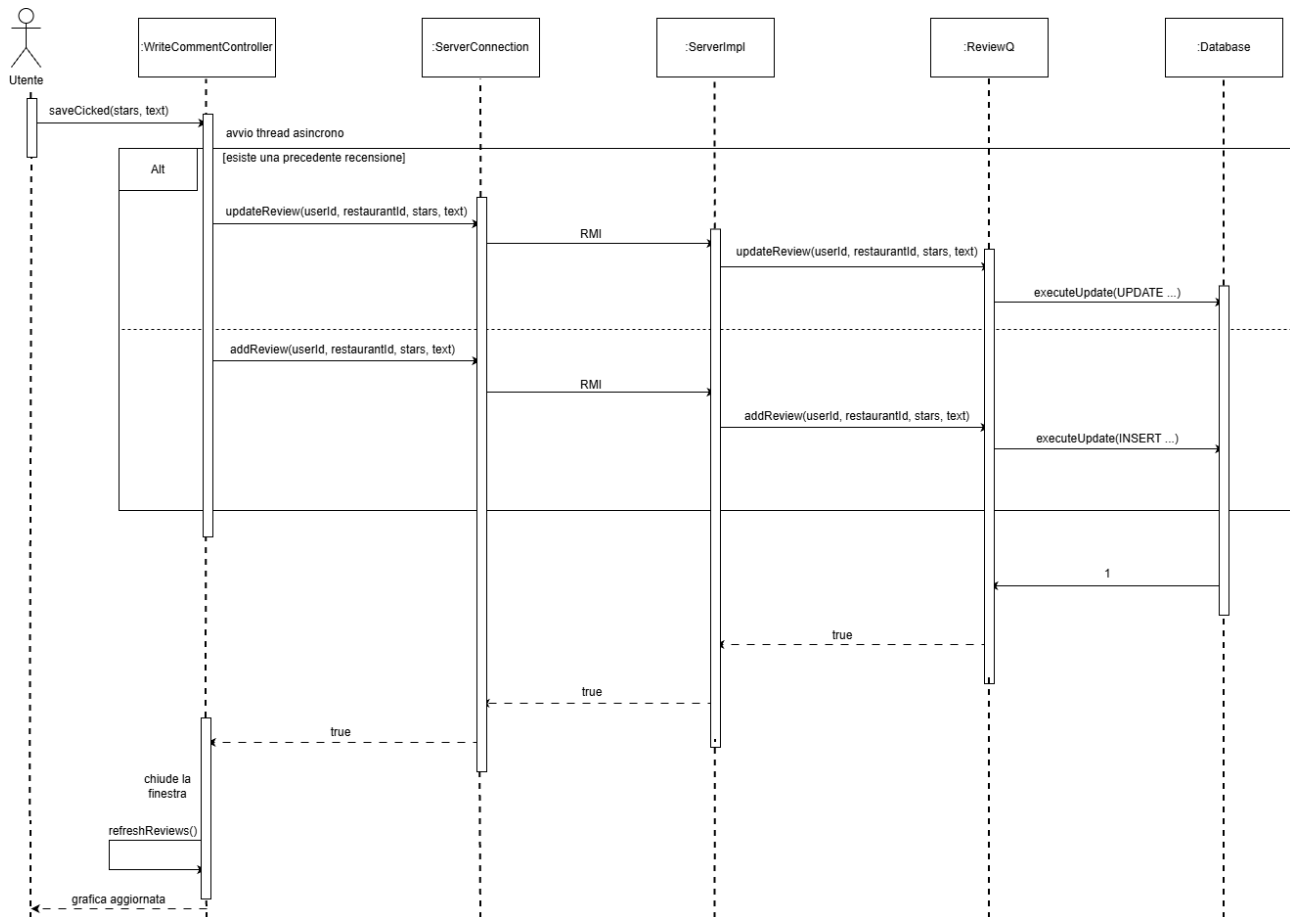
Nota: non sono stati rappresentati tutte le classi controller in quanto presentano tutte la stessa struttura, ne sono state solo rappresentate le principali.





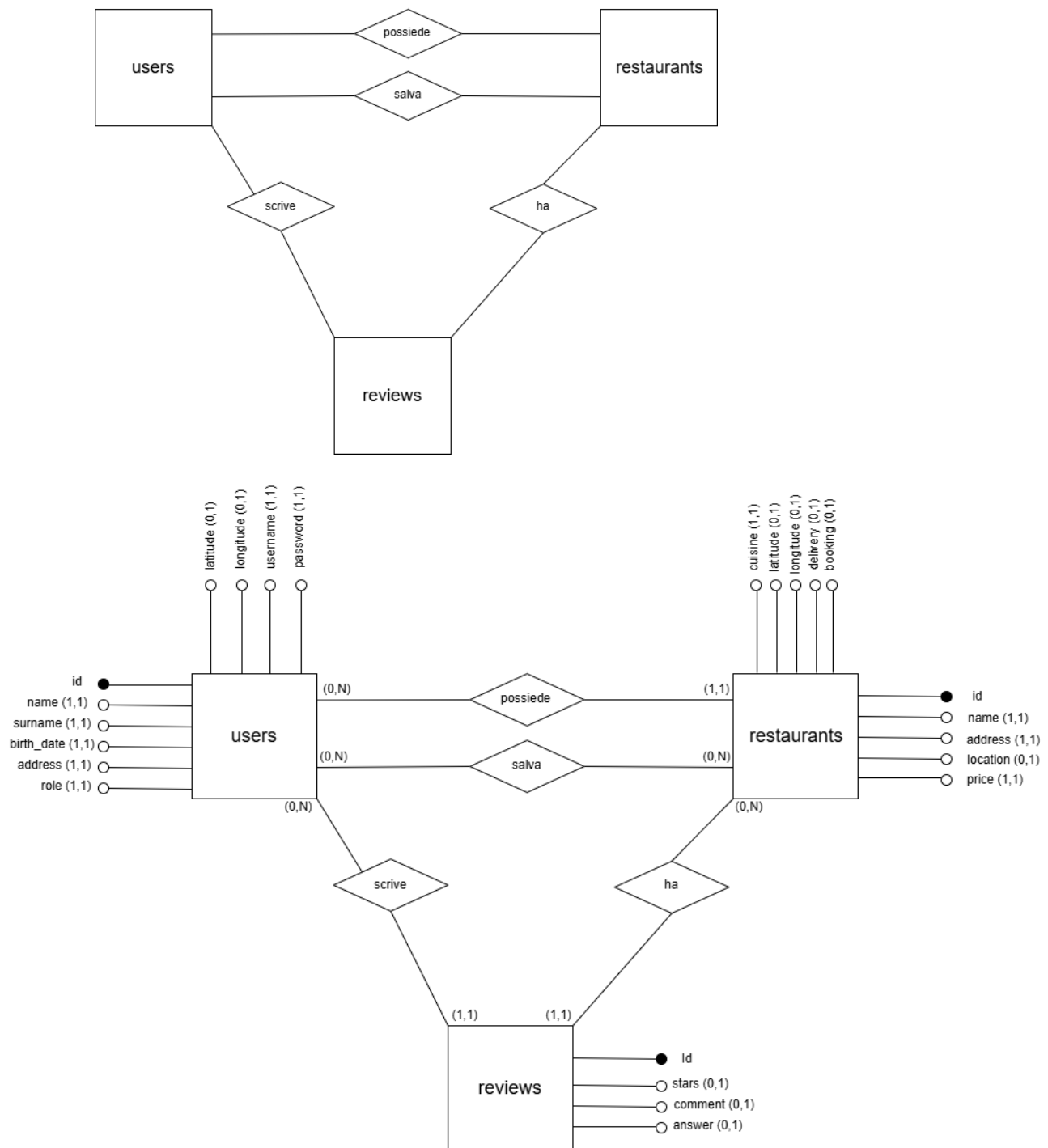
Mentre con i Sequence Diagram sono stati evidenziati i messaggi che vengono scambiati tra Client e Server quando l'utente richiede una specifica funzione, la cui logica è riassunta all'interno dello Use Case Diagram. Nella figura sottostante viene rappresentata la richiesta dell'utente per eseguire la pubblicazione di una recensione: una volta cliccato il tasto salva, il controller impacchetta gli input dell'utente (ricavando anche id dell'utente e del ristorante a cui vuole lasciare la recensione) e li spedisce al Proxy (ServerConnection), il quale si occuperà di mandare la richiesta al server. Una volta raggiunto il server, la richiesta verrà inoltrata alla classe ReviewQ, contenente le query relative alle recensioni, e invierà la query al database. Il database si occuperà di eseguire la query ricevuta e di ritornare una risposta, la quale seguirà il processo inverso fino al controller, il quale aggiornerà la grafica. Le possibili query eseguibili in questo caso sono due che dipendono dal fatto che l'utente abbia già recensito quel ristorante, e in caso si tratterà di un aggiornamento (update), oppure no e in questo caso si tratterà di un'aggiunta (insert).

Nota: abbiamo rappresentato solo questo caso tramite Sequence Diagram, poiché ci sono infinite interazioni eseguibili dall'utente e questo caso specifico rappresenta un metodo generale e ben descrittivo con cui vengono applicate le restanti funzioni.



DATABASE

È stato utilizzato il database relazionale PostgreSQL, come accennato precedentemente, per il salvataggio dei dati. Una volta creato e configurato, possiamo procedere con la descrizione delle tabelle utilizzate all'interno del programma. Come prima cosa è stato creato lo schema scheletro composto unicamente da entità (rappresentazioni delle collezioni di oggetti rilevanti nel dominio applicativo), e relazioni (legame logico tra entità). Successivamente è stato fatto lo schema ER (Entità Relazionale) dove sono stati aggiunti gli attributi (proprietà delle entità o associazioni) e le varie cardinalità (valori massimi o minimi che possono assumere gli attributi).



In base alle specifiche fornite, sono state individuate le seguenti entità:

- **Users:** contiene l'elenco di tutti gli utenti. È obbligatorio specificare nome, cognome, anno di nascita, indirizzo, ruolo, username e password. La latitudine e la longitudine vengono calcolati dal programma e inseriti successivamente. Ogni utente dispone di un id univoco, il quale permette molto facilmente di collegare un utente alle altre entità.

- **Restaurants:** contiene l'elenco di tutti i ristoranti. È obbligatorio specificare nome, indirizzo, fascia di prezzo e tipo di cucina. I restanti attributi vengono calcolati successivamente, oppure sono facoltativi (come la possibilità di fare prenotazioni o eseguire consegne a domicilio).
- **Reviews:** contiene l'elenco di tutte le recensioni scritte dagli utenti.

Oltre alle entità, sono state individuate le seguenti relazioni nello schema ER:

- **Possiede:** un utente può possedere nessun ristorante oppure n ristoranti, e ogni ristorante è posseduto da un utente.
- **Salva:** un utente può salvare tra i preferiti nessuno o N ristoranti, mentre un ristorante può essere tra i preferiti di nessun utente o di N utenti.
- **Scrive:** un utente può scrivere nessuna o N recensioni, mentre una recensione può essere scritta da un solo utente.
- **Ha:** un ristorante ha nessuna o N recensioni, mentre una recensione può appartenere ad un solo ristorante.

QUERY UTILIZZATE

Nota: con il punto interrogativo “?” vengono evidenziati i valori soggetti a cambiamenti in base all'utente.

Popolamento

- **Creazione tabella utenti**
CREATE TABLE IF NOT EXISTS users (
 id SERIAL PRIMARY KEY,
 name varchar(50) NOT NULL,
 surname varchar(50) NOT NULL,
 birth_date date NOT NULL,
 address varchar(255) NOT NULL,
 latitude double precision,
 longitude double precision,
 username varchar(50) NOT NULL UNIQUE,
 password varchar(255) NOT NULL,
 role varchar(20) NOT NULL

);

- **Creazione tabella ristoranti**

```
CREATE TABLE IF NOT EXISTS restaurants (  
    id SERIAL PRIMARY KEY,  
    owner_id integer REFERENCES users(id) ON DELETE CASCADE,  
    name varchar(255) NOT NULL,  
    address varchar(255) NOT NULL,  
    location varchar(255),  
    price varchar(10) NOT NULL,  
    cuisine varchar(255) NOT NULL ,  
    latitude double precision,  
    longitude double precision,  
    delivery varchar(50),  
    booking varchar(255)  
);
```

- **Creazione tabella preferiti**

```
CREATE TABLE IF NOT EXISTS bookmarks (  
    user_id integer NOT NULL REFERENCES users(id) ON DELETE  
CASCADE,  
    restaurant_id integer NOT NULL REFERENCES restaurants(id) ON  
DELETE CASCADE,  
    PRIMARY KEY(user_id, restaurant_id)  
);
```

- **Creazione tabella recensioni**

```
CREATE TABLE IF NOT EXISTS reviews (  
    id SERIAL PRIMARY KEY,  
    user_id integer REFERENCES users(id) ON DELETE CASCADE,  
    restaurant_id integer REFERENCES restaurants(id) ON DELETE  
CASCADE,  
    stars integer,  
    comment text,  
    answer text,  
    CONSTRAINT reviews_stars_check CHECK (((stars >= 1) AND (stars <=  
3)))  
);
```

Classe UserQ

- **Accesso**

```
SELECT id, username, password, role, address, latitude, longitude
FROM users
WHERE username = ?
```

- **Registrazione**

```
INSERT INTO users (name, surname, birth_date, address, latitude,
longitude, username, password, role)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
```

Classe RestaurantQ

- **Verifica se il database ha ristoranti o meno**

```
SELECT COUNT(*) AS total
FROM restaurants
```

- **Ristoranti nelle vicinanze**

```
SELECT restaurants.id, restaurants.name, restaurants.address,
restaurants.cuisine, restaurants.price, restaurants.delivery,
restaurants.booking, " + haversineFormula() + " AS distance,
COALESCE(AVG(reviews.stars), 0) AS avg_stars, COUNT(reviews.id) AS
total_reviews
```

```
FROM restaurants
```

```
LEFT JOIN reviews ON restaurants.id = reviews.restaurant_id
```

```
GROUP BY restaurants.id, restaurants.name, restaurants.address,
restaurants.cuisine, restaurants.price, restaurants.delivery,
restaurants.booking, restaurants.latitude, restaurants.longitude
```

```
ORDER BY distance ASC
```

```
LIMIT 10
```

- **Ristoranti nelle preferiti**

```
SELECT restaurants.id, restaurants.name, restaurants.address,
restaurants.cuisine, restaurants.price, restaurants.delivery,
```

```
restaurants.booking, " + haversineFormula() + " AS distance,  
COALESCE(AVG(reviews.stars), 0) AS avg_stars, COUNT(reviews.id) AS  
total_reviews
```

```
FROM restaurants
```

```
INNER JOIN bookmarks ON restaurants.id = bookmarks.restaurant_id
```

```
LEFT JOIN reviews ON restaurants.id = reviews.restaurant_id
```

```
WHERE bookmarks.user_id = ?
```

```
GROUP BY restaurants.id, restaurants.name, restaurants.address,  
restaurants.cuisine, restaurants.price, restaurants.delivery,  
restaurants.booking, restaurants.latitude, restaurants.longitude
```

```
ORDER BY distance ASC
```

- **Ristoranti recensiti**

```
SELECT restaurants.id, restaurants.name, restaurants.address,  
restaurants.cuisine, restaurants.price, restaurants.delivery,  
restaurants.booking, " + haversineFormula() + " AS distance,  
COALESCE(AVG(reviews.stars), 0) AS avg_stars, COUNT(reviews.id) AS  
total_reviews
```

```
FROM restaurants
```

```
INNER JOIN reviews my_rev ON restaurants.id = my_rev.restaurant_id LEFT  
JOIN reviews ON restaurants.id = reviews.restaurant_id
```

```
WHERE my_rev.user_id = ?
```

```
GROUP BY restaurants.id, restaurants.name, restaurants.address,  
restaurants.cuisine, restaurants.price, restaurants.delivery,  
restaurants.booking, restaurants.latitude, restaurants.longitude
```

```
ORDER BY distance ASC
```

- **Ricerca ristoranti e filtro ricerca**

```
SELECT      restaurants.id,      restaurants.name,      restaurants.address,  
restaurants.cuisine,      restaurants.price,      restaurants.delivery,  
restaurants.booking, " + haversineFormula() + " AS distance,
```



```
COALESCE(AVG(reviews.stars), 0) AS avg_stars, COUNT(reviews.id) AS
total_reviews

FROM restaurants

LEFT JOIN reviews ON restaurants.id = reviews.restaurant_id

WHERE 1=1

AND (restaurants.address ILIKE ? OR restaurants.address ILIKE ? OR
restaurants.address ILIKE ? OR restaurants.address ILIKE ? OR
restaurants.address ILIKE ?)

AND restaurants.cuisine ILIKE ?

AND restaurants.price = ?

AND restaurants.delivery IS NOT NULL AND restaurants.delivery != ''

AND (restaurants.delivery IS NULL OR restaurants.delivery = '')

AND restaurants.booking IS NOT NULL AND restaurants.booking != ''

AND (restaurants.booking IS NULL OR restaurants.booking = '')

GROUP BY restaurants.id, restaurants.name, restaurants.address,
restaurants.cuisine, restaurants.price, restaurants.delivery,
restaurants.booking, restaurants.latitude, restaurants.longitude

HAVING COALESCE(AVG(reviews.stars), 0) >= ?

ORDER BY distance ASC LIMIT 10 OFFSET ?
```

- **Propri ristoranti**

```
SELECT restaurants.id, restaurants.name, restaurants.address,
restaurants.cuisine, restaurants.price, restaurants.delivery,
restaurants.booking, 0.0 AS distance, COALESCE(AVG(reviews.stars), 0) AS
avg_stars, COUNT(reviews.id) AS total_reviews
FROM restaurants
LEFT JOIN reviews ON restaurants.id = reviews.restaurant_id
WHERE restaurants.owner_id = ?
GROUP BY restaurants.id, restaurants.name, restaurants.address,
restaurants.cuisine, restaurants.price, restaurants.delivery,
restaurants.booking, restaurants.latitude, restaurants.longitude
```

ORDER BY restaurants.id DESC

- **Aggiungere un ristorante**

INSERT INTO restaurants (name, address, cuisine, price, delivery, booking, latitude, longitude, owner_id)

VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)

- **Rimuovere un ristorante**

DELETE FROM restaurants WHERE id = ?

Classe BookmarkQ

- **Controllare se è tra i preferiti**

SELECT 1 FROM bookmarks

WHERE user_id = ? AND restaurant_id = ?

- **Aggiungere preferito**

INSERT INTO bookmarks (user_id, restaurant_id)

VALUES (?, ?)

- **Rimuovere preferito**

DELETE FROM bookmarks

WHERE user_id = ? AND restaurant_id = ?

Classe ReviewQ

- **Recensioni di un ristorante**

SELECT users.name, reviews.stars, reviews.comment, reviews.answer
FROM reviews

JOIN users ON reviews.user_id = users.id

WHERE reviews.restaurant_id = ?

- **Recensioni utente**

SELECT stars, comment, answer
FROM reviews

WHERE user_id = ? AND restaurant_id = ?

- **Aggiungere recensione**

INSERT INTO reviews (user_id, restaurant_id, stars, comment)

VALUES (?, ?, ?, ?)

- **Aggiornare recensione**

UPDATE reviews SET stars = ?, comment = ?

```
WHERE user_id = ? AND restaurant_id = ?
```

- **Rimuovere recensione**

```
DELETE FROM reviews
```

```
WHERE user_id = ? AND restaurant_id = ?
```

- **Recensioni di un proprio ristorante**

```
SELECT users.name, reviews.stars, reviews.comment, reviews.answer,  
reviews.user_id
```

```
FROM reviews
```

```
JOIN users ON reviews.user_id = users.id
```

```
WHERE reviews.restaurant_id = ?
```

- **Rispondere a una recensione**

```
UPDATE reviews SET answer = ?
```

```
WHERE user_id = ? AND restaurant_id = ?
```

SCELTE ARCHITETTURALI

L'architettura del Client è stata progettata seguendo il design pattern Model-View-Controller (MVC), supportato nativamente dal framework JavaFX. Questa scelta garantisce una separazione delle responsabilità, disaccoppiando la logica di presentazione da quella applicativa e di comunicazione di rete.

GESTIONE DELLA CONCORRENZA

Per la gestione della concorrenza dell'applicazione, non è stata implementata una logica propria perché la JDBC (Java Database Connectivity), sfruttando l'auto-commit, rende ogni operazione fatta sul database permanente e gestisce autonomamente la concorrenza appoggiandosi alle proprietà ACID del database relazionale. Mentre per quanto riguarda la comunicazione distribuita, è stato utilizzato "Remote Method Invocation" (detto anche RMI), il quale si occupa autonomamente di gestire le richieste multiple assegnando un thread separato per ogni client. Questo è stato possibile perché tutto il progetto è stato

sviluppato utilizzando solo Java. In particolare, RMI permette al server (ServerImpl) di rendere un oggetto remoto e poi pubblicarlo su un Registry (un registro che associa un nome ad un oggetto remoto, in questo caso con il metodo rebind). In questo modo, il client può ricercare un riferimento a questo oggetto utilizzando solo il suo nome (nel nostro caso è theKnifeServer). Una volta ottenuto il riferimento, il client può invocare i metodi definiti nell'interfaccia remota dell'oggetto (ServerInterface), potendo così utilizzare le funzionalità che offre il server. Così facendo il client non ha bisogno di conoscere i dettagli implementativi del server.

LIMITI DELLA SOLUZIONE SVILUPPATA

Per il momento la nostra soluzione utilizza un database locale e in futuro si potrebbe implementare una soluzione cloud basata su database PostgreSQL. Inoltre, l'applicazione ha una toolbar propria, permette di trascinare la finestra e ha un design unico, ma non permette ancora di ridimensionare la finestra.